

Design Stock Exchange - A Deep Dive

#107: System Design Stock Exchange - Part 3



NEO KIM

DEC 11, 2025 PAID



- *[Share this post](#) & I'll send you some rewards for the referrals.*
- *Block diagrams created with [Eraser](#).*

Last month, I launched **Design, Build, Scale**... the newsletter series that will elevate your software engineering career.

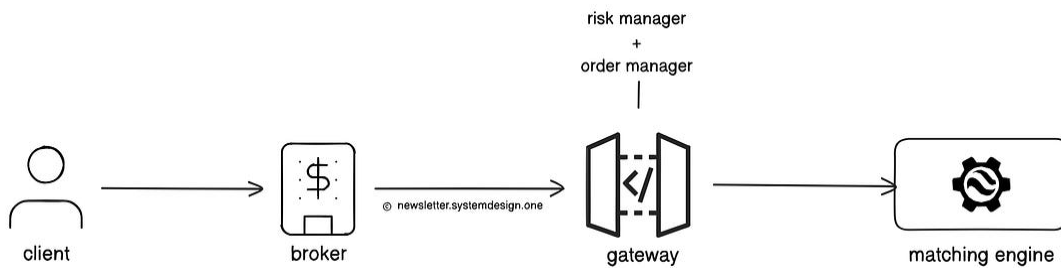
This email (Part 3) is a little longer than the previous two, so I'll get straight into it...

TL;DR of [Part 1](#) & [Part 2](#)!

Key components of an exchange are:

- Broker - allows people to buy and sell stocks on the exchange.
- Gateway - entry point for brokers to send BUY or SELL orders to the exchange.

- Matching Engine - matches BUY and SELL orders to create trades.



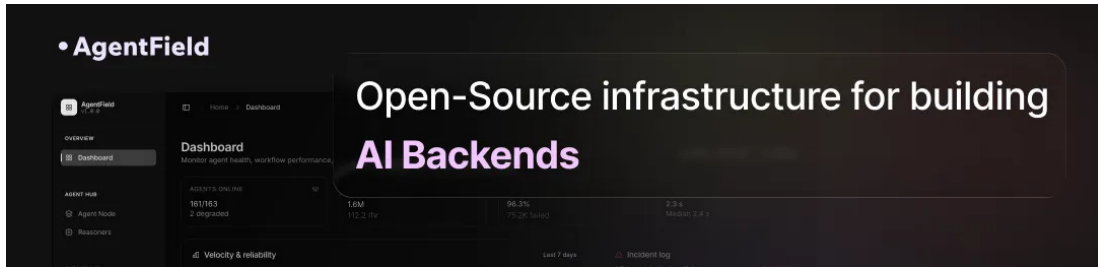
Order Processing Workflow:

1. User creates a BUY or SELL order through the broker.
2. Gateway validates the order: authentication, check request format & rate limiting.
3. Then it forwards the request to the order manager. It performs risk checks using rules defined in the Risk Manager.
4. Risk manager verifies whether the user's wallet has sufficient funds. Plus, it freezes the required amount in the wallet to prevent double-spending.
5. Order Manager sends the request to the Matching Engine.
6. Matching Engine keeps an in-memory "order book" for each stock. It matches BUY and SELL orders based on price and time priority.
7. After a match occurs, the Matching Engine creates two executions (fills). One for the buyer. One for the seller.
8. Matching Engine distributes trade results as market data to clients through the Gateway. It uses UDP multicast¹ for efficiency.

Most steps "before and after" matching run in parallel to BOOST throughput.

Onward.

Start building AI backends—Agents as microservices (Sponsor).



Chatbots and copilots are just the surface.

The real shift is happening underneath, where AI agents work behind the scenes, making smart decisions that developers used to hardcode.

AgentField is the open-source control plane for that:

- Run agents as services
- Discover services/agents
- Coordinate tasks asynchronously
- Use cryptographic IAM and audit trails

Use it to build AI-native backends that can reason about orders, customers, payments, and incidents (while following the rules YOU set).

You focus on the logic. AgentField handles running it in production.

1. **Explore the code on GitHub:** <https://github.com/Agent-Field/agentfield>
 - Try examples, test features, and ★ the repo if it helps!
2. **New to AI backends?** Start here:
<https://www.agentfield.ai/blog/posts/ai-backend>
 - See why autonomous backends are crucial for system design.

3. **Want to run 1000s of agents?** Read this: <https://agentfield.ai/docs>

- Learn how to run agents safely at scale.

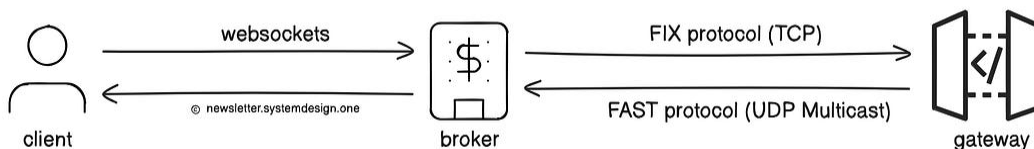
★ STAR THE REPO

FIX Protocol

Broker send orders to the gateway using the Financial Information eXchange (**FIX**) protocol (for institutional clients).

FIX is a text-based protocol. Although reliable, it's slow because text parsing consumes CPU. Gateway uses a FIX engine to receive and send FIX messages.

You can build your own engine for ultra-low latency, or buy a pre-optimized commercial one.



Exchange streams market data using FIX Adapted for Streaming² (**FAST**).

It's much faster because it's a binary, compressed version of FIX... binary encoding is compact and easy for CPUs to parse.

Many exchanges provide even faster binary protocols for high-frequency trading:

- Integrated Trade Capture Handler (**ITCH**) - a fast market data feed

- Order Unified Communication Handler (**OUCH**) - a fast order entry protocol

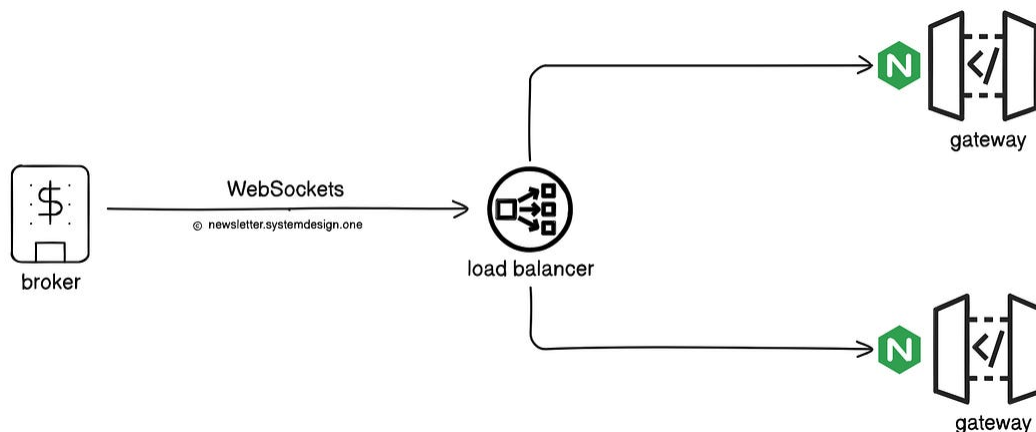
These avoid text parsing and are specifically designed for low-latency trading.

Let's keep going!

Gateway

It accepts connections from brokers and supports REST + WebSocket APIs³ (for retail traders).

Gateway could use Nginx server for HTTPS termination and WebSocket routing. Plus, exchange could run many gateway servers in parallel behind a load balancer for high availability.



Gateway:

- Tracks sequence numbers of each client,
- Normalize and validate incoming messages,

- Protects matching engine from malformed or abusive client traffic,
- Uses TCP backpressure⁴ to slow down misbehaving or overloaded clients automatically,
- Allows only ONE outstanding order per client to maintain strict, deterministic ordering.

Exchanges used to have MANY gateways (some faster, some slower).

But traders exploited this by:

- Flooding slower gateways with junk orders to delay competitors,
- Routing orders only through the fastest gateway (unfair advantage).

So modern exchanges use a SINGLE gateway for **all clients of the “same class”**.



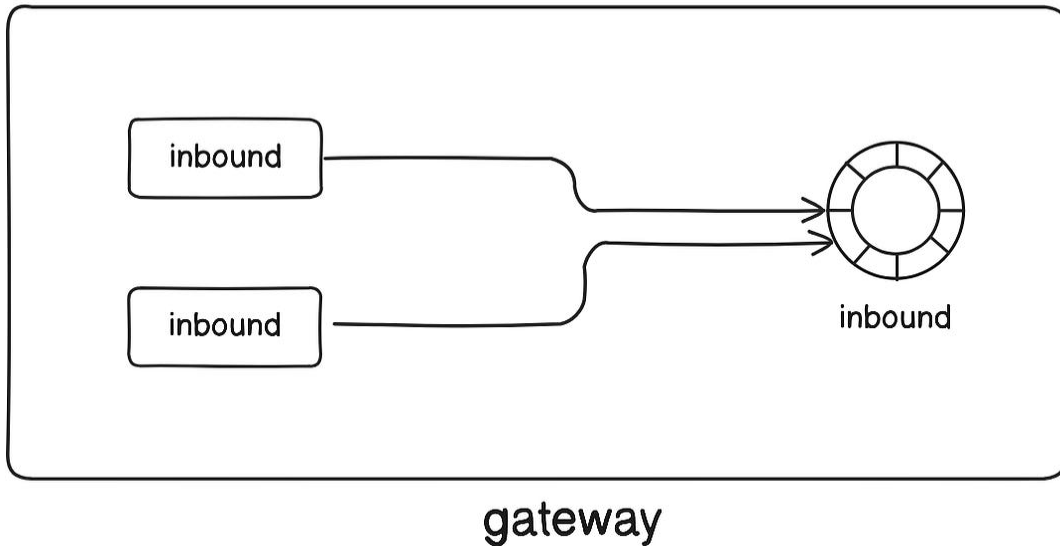
This means all clients in the same class (institutional clients, retail traders) have identical latency⁵. Thus ensuring fairness and reducing system load.

Here's how Gateway handles concurrency:

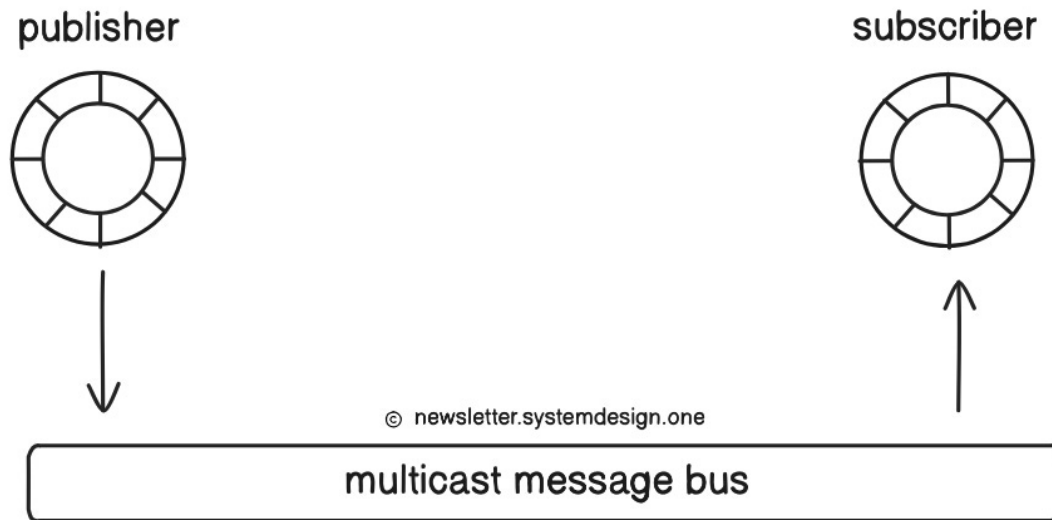
- Inbound (broker → gateway):
 - Assigns one thread per TCP connection
 - Parses messages and publishes them to a Disruptor ring buffer

- Disruptor efficiently distributes events to internal services (order manager, risk checks,...)

© newsletter.systemdesign.one



- Outbound (matching engine → broker):
 - Outbound events come through the ring buffer
 - Ring buffer dispatches events to subscribed services
 - Each session has its own outbound handler thread
 - If a client is slow, backpressure (ring buffer fills up) prevents system slowdowns



NOTE: Inbound and outbound paths share no mutable state.

Wallet

A wallet stores a user's funds & assets for trading.

But there's a risk of DOUBLE SPENDING if two orders from the same user arrive at once... and both threads try to read the "same" balance.

Example:

- Thread A reads balance X and subtracts Y
- Thread B reads balance X and subtracts Z

Here are three ways to prevent this problem:

1 Mutex Locks

Only one thread updates the user's balance at a time. This is safe, but it significantly slows down the system.

2 ConcurrentHashMaps

They allow fast (parallel) reads/writes if keys fall into different segments. Yet it doesn't protect the correctness of multi-step operations (read → modify → write).

3 Database Row-Level Locking

Exchanges rely on SQL databases (like MySQL InnoDB) because they:

- Support ACID (safe transactions)
- Provide row-level locking, so two threads cannot update the same user's balance at once

Plus, this approach makes the read → modify → write operations safer.

Sample Database Schema

- user_account (user_id, balance)
- wallet (user_id, symbol, quantity)
- stock (symbol, price)

Tables can be sharded by "user_id" for performance.

Data Structures

- account balance - HashMap {user_id: balance}
- stock price - HashMap {symbol: price}

Ready for the next technique?

Disruptor Pattern

Disruptor is a concurrency model that doesn't use locks.

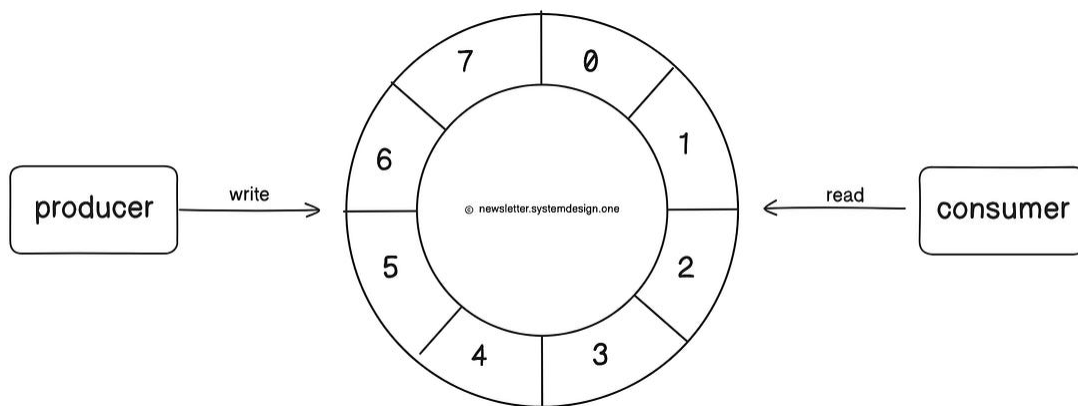
It's built as a ring buffer (fixed-size circular array with up to 20 million slots).

Imagine the ring buffer as:

"super-fast, in-memory queue for passing events between threads."

Producers write events; consumers read them in order.

Each thread maintains its own sequence number, so there is no locking, blocking, or contention.



A disruptor offers:

- high throughput,
- microsecond-level latency.

It's excellent for workloads where many components must process the same event.

Here's WHERE the stock exchange uses the disruptor:

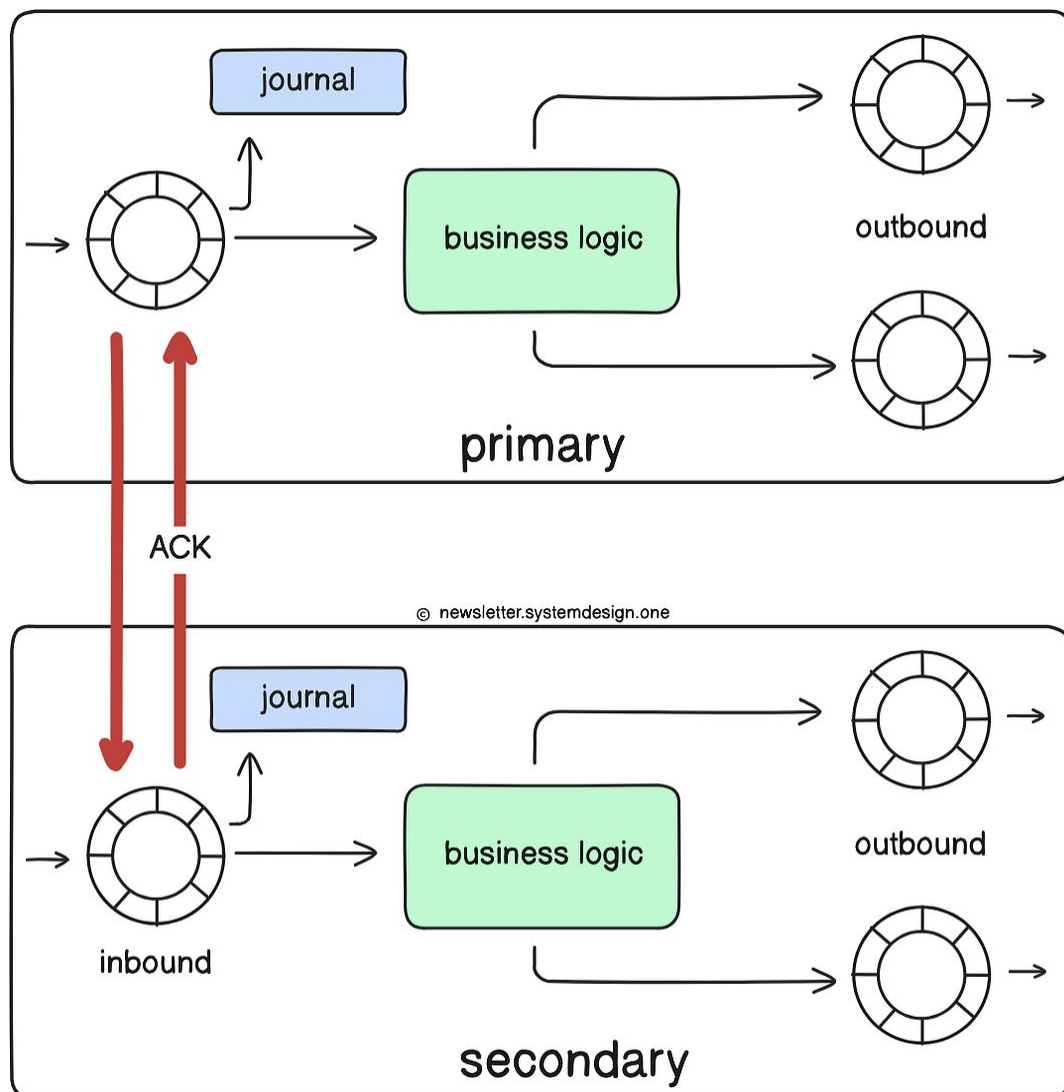
1 Gateway

Gateway publishes incoming orders to an input disruptor.

It allows many tasks to run in parallel (without blocking each other), such as:

- journaling to disk
- unmarshalling (parsing messages)
- assigning global sequence numbers
- replicating events to standby engines

Each step is independent; disruptor provides high throughput without locks.



2 Order Manager

Order manager uses disruptors to broadcast events to various internal consumers, such as:

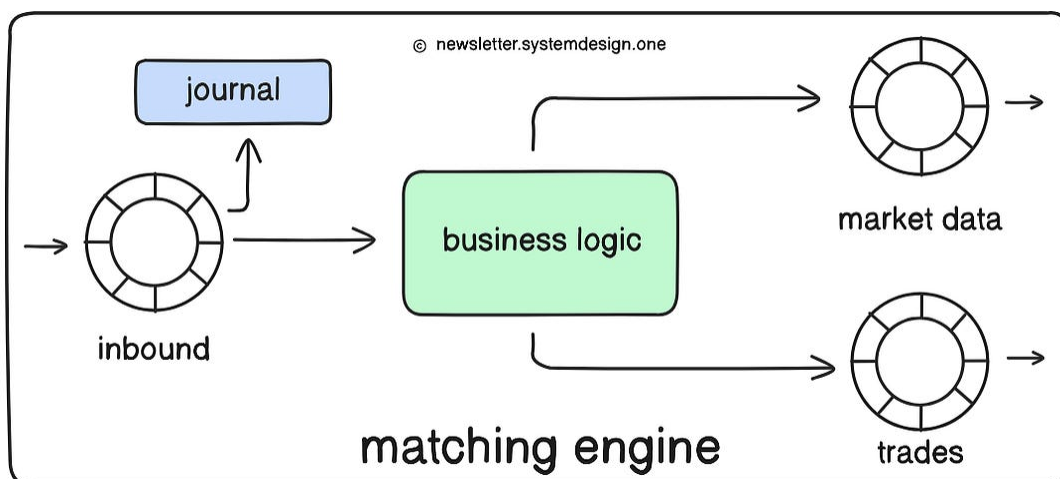
- risk checks
- routing logic
- audit/journal writers
- user balance service
- sequence enforcement

Each consumer reads using its own sequence counter,,, so they never block each other.

3 Market Data

Output disruptors take results and prepare them for network output:

- execution reports
- market data updates
- client notifications via gateway



Each topic (e.g., trades, market data) gets its own disruptor to avoid interference.

Ready for the best part?

Reminder: this is a teaser of the subscriber-only newsletter series, exclusive to my golden members.



When you upgrade, you'll get:

- **High-level architecture of real-world systems.**
- Deep dive into how popular real-world systems actually work.
- **How real-world systems handle scale, reliability, and performance.**

✓ Subscribed

Matching Engine

Matching engine receives orders, matches them, and outputs trades.

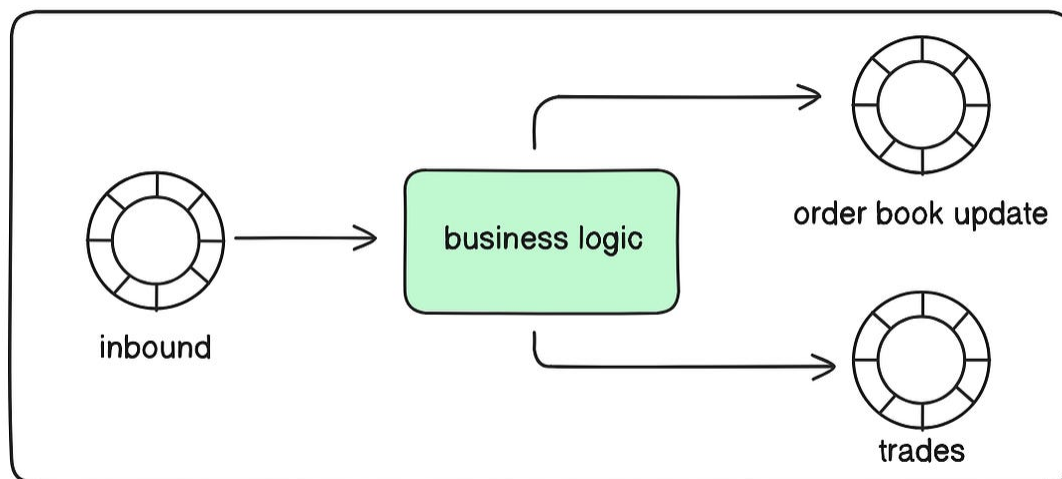
i.e., it runs all trading logic (business logic processor).

Modern exchanges process 5 million orders per second, with an end-to-end latency (99th percentile) of less than 100 MICROSECONDS⁶.

Here's how:

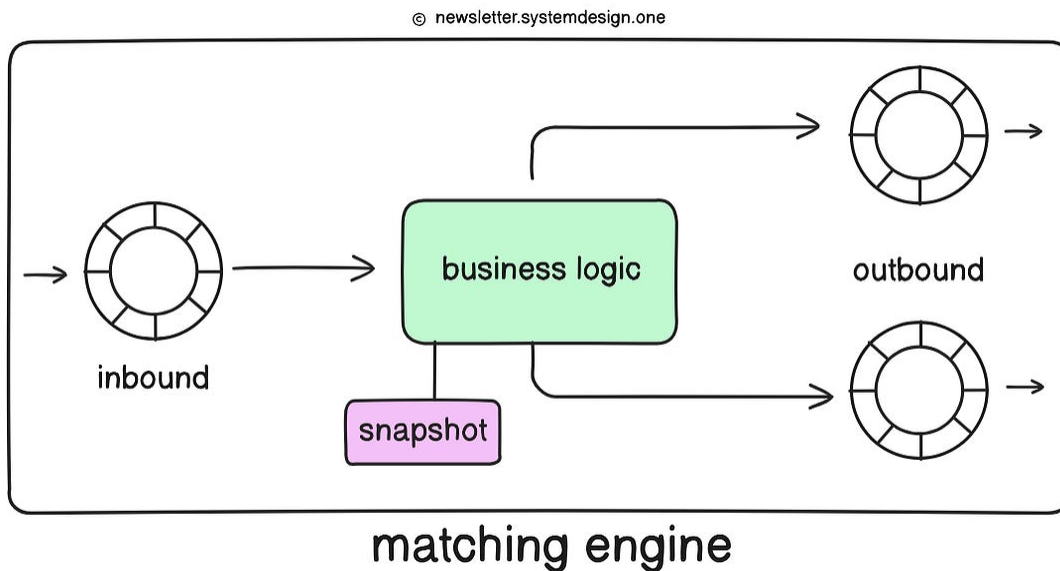
- Uses Java, C++, Rust, or Golang for high performance.
- Process each stock in a single thread for deterministic matching, avoid locking, and achieve high performance.
- Stateful and stores everything "in-memory" for speed:
 - No database queries (instead, simply programming).
 - All changes come from events, not from external calls.
 - Business logic gets written as plain Java objects in memory.
- Caches on ONE CPU core for performance.

© newsletter.systemdesign.one



matching engine

- Uses disruptor pattern to handle inbound and outbound traffic.
 - Inbound disruptor provides reliability for incoming orders during burst traffic⁷.
 - Outbound topics (e.g., order book updates, trades) get their own queue and thread. There's a separate thread to consume events per topic.
- Moves anything non-critical (e.g., *"cancel after 3 minutes"*) to helper services.
- Journals events to disk, so it can rebuild its state by replaying events after a crash.
- Regularly snapshots its state (as a binary file) for a quick restart.



- Sends all events to a hot standby for failover (fault tolerance). Only primary server sequences events; standby mirrors.
 - Replicates to another data center (without ACK for low latency).
-

Know someone who wants to learn system design? Consider gifting a subscription:

[Give a gift subscription](#)

Order Book

Order book tracks all open BUY and SELL orders that haven't been filled yet.

It's just a "storage" to organize orders so matching engine can find:

- best BUY price
- best SELL price

Let's look at different ways to set up an order book:

1 Priority Queue

Use red-black trees (balanced binary search trees) to implement priority queues.

It stores an order in TWO trees at the same time:

- One tree sorted by price: match orders quickly by finding the best price
- Another tree sorted by order ID: find or cancel a specific order quickly

As same order node links into both trees, you get a direct pointer to remove it from the other when you remove it from one.

It's even possible to use THREE trees:

1. BUY orders sorted by price
2. SELL orders sorted by price
3. ALL orders sorted by order ID

This allows:

- finding best prices fast,
- matching orders quickly,
- canceling orders instantly,
- avoid scanning the entire order book.

But the worst-case time complexity is $O(n \log n)$. So it slows down under heavy load.

2 LiquiBook

It's used as a "reference" by many modern exchanges.

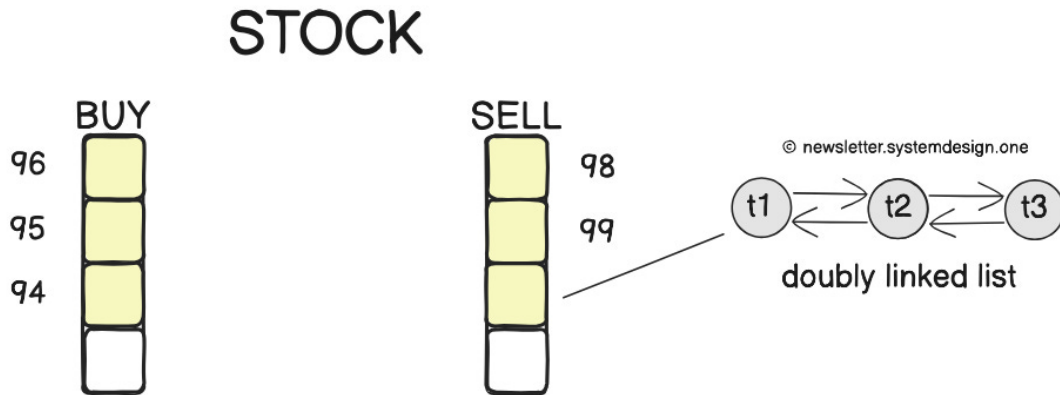
LiquiBook keeps an in-memory list of open orders for each stock (symbol).

There are actually TWO lists for each stock:

- One for BUY orders,
- Another one for SELL orders.

Each list gets sorted by price and timestamp in a first-in, first-out (FIFO) manner.

It keeps track of the highest bid and lowest ask for quick matching. For example, BUY@96 and SELL@98.



Also each price level gets a separate queue of orders (doubly linked list).

(More on this later...)

Order book decides nothing. Instead, it just holds orders.

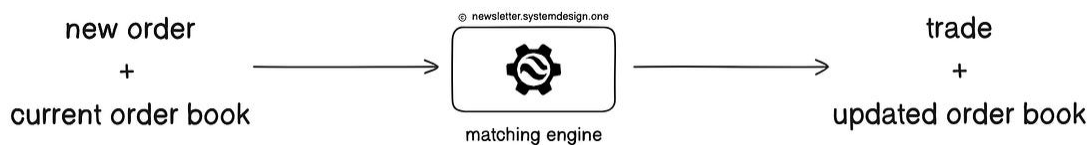
So who makes the matching decisions?

Matching Logic

Matching logic is the core algorithm (decision-making part).

It's single-threaded (one thread per stock symbol) to ensure deterministic, fair ordering.

Think of it as a pure function (functional programming):



This logic sits on top of the order book and decides WHEN and HOW trades occur.

Now let's look at different ways to implement matching logic:

1 Priority Queue

Store orders in heaps (also called priority queues).

- SELL orders go in a min-heap, so the lowest sell price is at the top
- BUY orders go in a max-heap, so the highest buy price is at the top

Here's how matching a BUY order works:

1. Look at the root of the SELL min-heap (best price)
2. It's a match if $\text{sell_price} \leq \text{buy_price}$
3. Remove orders from the heaps

But it becomes slow under high load, with a worst-case time complexity of $O(n \log n)$.

NOTE: worst-case time complexity is higher than a simple linear search in $O(n)$.

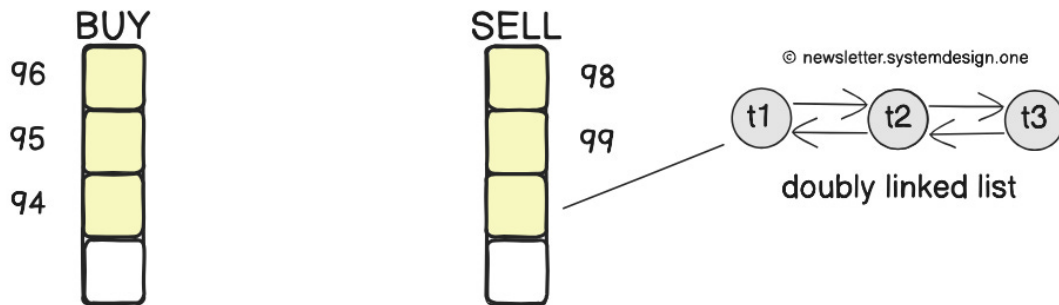
So real exchanges don't use this approach. This method is helpful ONLY for academic or toy implementations.

2 FIFO Price-Level (LiquiBook)

Each stock symbol keeps:

- A sorted map of BUY price levels
- And a sorted map of SELL price levels
- Each price level stores orders in a FIFO queue (doubly linked list)

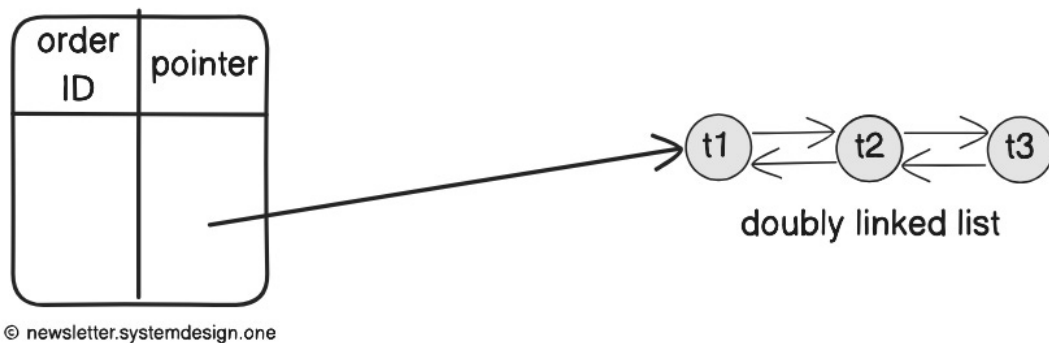
STOCK



Yet searching through all price levels and linked lists could be slow - $O(n)$.

So they use an index to map order IDs to the order object in memory:

Order ID → pointer (reference)



This allows fast removal of "canceled or filled orders" by looking up the order by ID in the index in $O(1)$. Then setting its amount to 0.

A closed order gets removed from the order book and gets added to the transaction history.

Here's how adding a new order works:

1. Find the price level to which the new order belongs
2. Put it at the end of that level's queue - $O(1)$ time complexity
3. Update the best bid/ask if needed
4. Add its pointer to a fast lookup table (OrderIndex)

Here's how matching a BUY order works:

1. Get the best SELL price (lowest) from the sorted map
2. It's a match if $\text{sell_price} \leq \text{buy_price}$
3. Walk through the orders at that price level and fill them one by one
4. If the BUY order remains, move to the next higher SELL price level
5. Repeat (1-3) until:
 - Order gets fully filled,
 - No SELL orders at or below the buyer's price.
6. If partially filled, insert the rest into the correct BUY price level's queue

NOTE: Fill = $O(k)$; k = number of price levels touched.

Here's how canceling an order works:

1. Use HashMap to find the order in $O(1)$
2. Mark its quantity = 0
3. Remove it from the price level queue in $O(1)$

NOTE: Orders with quantity = 0 get skipped to speed up cancellation.

Background jobs can clean up expired or inactive orders...

Real exchanges use the FIFO price-level approach. It preserves price-time priority, is memory-efficient, and matches in $O(1)$ time within a

price level.

How Orders Behave

Here's how an order behaves with:

1 Partially filled order

If an order is ONLY partially matched, whatever is left stays in the order book as a resting order.



2 No match

If nothing can match it, the entire order gets added to the order book.

3 Many orders at the same price

If several orders match, the matching engine uses the chosen rule (FIFO or pro rata) to determine which order gets filled first.

Next, let's look at how a trade is split among eligible resting orders...

Know someone who wants to learn system design? Consider gifting a subscription:

Give a gift subscription

Allocation Algorithm

Allocation happens after the matching logic determines the eligible orders.

1 FIFO (Time-Price Priority)

This is the most common allocation rule.

- If many orders sit at the same price, they're filled in order of arrival
- But if an order gets modified, it loses its time priority

Once matching logic finds price-compatible orders, FIFO decides the fill order.

2 Pro-Rata

Bigger orders get a bigger share.

- It splits incoming orders proportionally to resting order size
- Price, size, and time also influence the split (time may matter less than size)

But traders might abuse the pro-rata algorithm... by placing large "fake" orders to gain priority, then canceling them quickly. So matching rules must detect and prevent such traders.

TL;DR:

- **Order book:** groups orders by price and time.
- **Matching logic:** finds which resting orders can trade (price match).

- **Allocation algorithm:** decides how much each resting order receives.

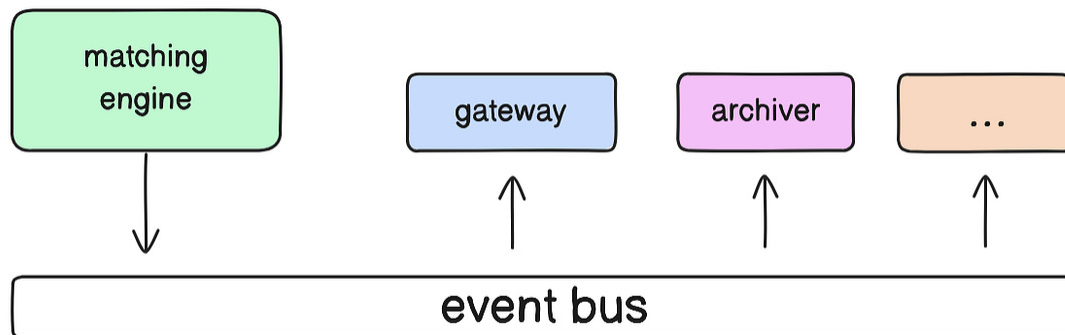
Market Data

Market data is “real-time” information about trades... it shows what’s currently happening in the market.

Here’s how it works:

1 Internal Event Buses (Inside the Exchange)

Exchange publishes events (Order, Trade, Account updates) to internal event buses.



© newsletter.systemdesign.one

Each user session has an actor⁸ that subscribes only to the data the user needs. So when an event arrives, the actor forwards it to the user via WebSocket.

2 High-Speed Internal Messaging (UDP Multicast)

A message queue, such as Kafka or RabbitMQ, provides ordering and durability.

But they add latency and overhead for fan-out under heavy load. So exchange uses UDP multicast to send a single packet to many receivers at once... i.e., internal fan-out of market data and system events.

UDP multicast keeps latency extremely low and ensures fairness.

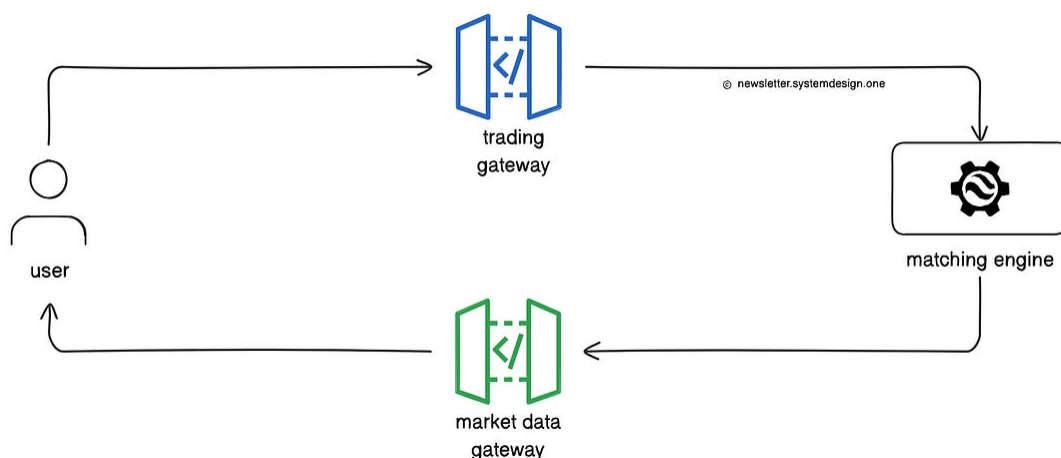
Yet UDP has its drawbacks:

- No ordering
- No delivery guarantee

So here's how they add reliability:

- Include a sequence number for messages so recipients can detect missing packets.
- Keep messages on retransmission servers for a short period; clients request a missing one if they detect a gap.
- Use libraries like Informatica UMS⁹ to add reliability while preserving microsecond latency.

3 External Market Data Delivery to Traders



- External clients (brokers, institutions) receive updates through UDP multicast feeds.
 - Multicast ensures all customers receive the same market data at the same time.
 - Retransmission servers allow clients to recover missing packets¹⁰ caused by UDP drops.
 - Keep separate gateways for inbound and outbound traffic for performance.
-

Low Latency Infrastructure

1 RDMA (Remote Direct Memory Access)

A server can read or write to another server's memory directly using RDMA.

It bypasses the operating system and CPU,,, thus making data transfer much faster and predictable than TCP/IP.

Exchanges use RDMA only inside the data center, for server-to-server communication, such as:

- syncing primary and secondary matching engines
- updating in-memory snapshots
- sending journal buffers to persistence services
- syncing risk engine or order-book state

Exchanges handle client traffic using:

- TCP (FIX/WebSocket)
- UDP multicast (market data)
- Proprietary binary feeds (cross-datacenter replication)

2 FPGA (Field Programmable Gate Array)

An FPGA is a programmable chip that can run custom logic at hardware speeds.

Exchanges use FPGAs for ultra-low-latency tasks that must run in “nanoseconds”, such as:

- parsing FIX messages
- applying risk checks
- market-data encoding
- sequencing inbound messages
- running on custom network cards (NICs) for fast packet handling

These components sit before the matching engine and speed up the critical path.

TAKEAWAYS

- Simple designs scale well.
 - Parallelism is good, but centralization has benefits.
 - Commodity hardware is sufficient if carefully designed.
-

I’ll add a little more information about scalability & performance in the next email.

Neo

PS - You know someone learning system design? Consider gifting a subscription:

Give a gift subscription



👋 Find me on [LinkedIn](#) & [Twitter](#)

References

- <https://www.janestreet.com/tech-talks/building-an-exchange/>
- <https://www.infoq.com/presentations/financial-exchange-architecture/>
- <https://www.infoq.com/presentations/lmax-trading-architecture/>
- <https://martinfowler.com/articles/lmax.html>
- <https://github.com/cyanly/gotrade>
- <https://github.com/enewhuis/liquibook>
- https://github.com/fmstephe/matching_engine
- <https://medium.com/lgogroup/a-matching-engine-for-our-values-part-1-795a29b400fa>
- <https://medium.com/lgogroup/a-matching-engine-for-our-values-part-2-60b71c9eef26>
- <https://www.infoq.com/news/2015/03/ddd-trading-example/>
- <https://www.infoq.com/interviews/Architecture-Eric-Evans-Interviews-Greg-Young/>

- <https://www.infoq.com/news/2016/04/event-sourcing-anti-pattern/>
- <https://www.infoq.com/presentations/messaging-web-trading/>
- <https://web.archive.org/web/20110219163448/http://howtohft.wordpress.com/2011/02/15/how-to-build-a-fast-limit-order-book/>
- <https://bhomnick.net/building-a-simple-limit-order-in-go/>
- <https://around25.com/blog/building-a-trading-engine-for-a-crypto-exchange/>
- <https://www.infoq.com/presentations/financial-services-languages-environments/>
- https://en.wikipedia.org/wiki/Reliable_User_Datagram_Protocol
- https://en.wikipedia.org/wiki/NACK-Oriented_Reliable_Multicast
- <https://www.cmegroup.com/confluence/display/EPICSANDBOX/Supported+Matching+Algorithms>

-
- 1 UDP multicast sends a single network packet to a special multicast address, and the network delivers it to many receivers at once without creating separate copies for each listener.
 - 2 The exchange encodes market data using FAST. Then it broadcasts these FAST-encoded packets over UDP multicast. Multicast delivers a single packet to thousands of subscribers simultaneously, and FAST keeps each packet small and fast to decode, reducing bandwidth and CPU load.
 - 3 FIX is for institutional members and brokers. WebSockets are for retail traders, mobile apps, and API users. Exchange supports both, so the gateway handles both protocols.
 - 4 TCP backpressure means the server can't keep up, so TCP forces the client to slow down by shrinking the receive window.
 - 5 Treated equally within each category. Gateways get separated by client category: retail, institutions, and co-located HFT.

- 6 This means 99% of trades are less than 100 microseconds.
- 7 Operating system kernel may drop UDP packets during bursts.
- 8 Disruptor Pattern handles latency-critical paths, while Actor Model handles paths where flexibility and concurrency are more important than speed.
- 9 Informatica UMS (aka 29 West LBM) can deliver up to 35 million 100-byte messages per second.
- 10 Messages contain sequence numbers. Consumers can request retransmission when they detect gaps.